



BITS Pilani
Pilani Campus

COMPILER DESIGN

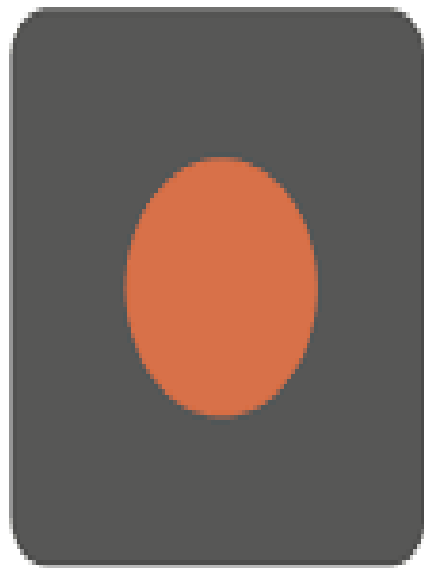
SSWT ZC373



Contact Session - 5

M3 – Syntax Analysis

IMP Note to Self



Start
Recording

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
 - Once you join the session, continue till the end to consider you as present in the class.
 - IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
 - **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**
-

Objectives



To Understand

- Top-down parsing
- LL(1) Grammars
- Implementing LL(1) Parser
- Examples on LL(1) Parser
- Parser Generator using YACC

Syntax Analysis and Parsing



1. *Syn-tax*

The way in which words are stringed together to form phrases, clauses and sentences

2. **Syntax analysis:**

The task concerned with fitting a sequence of tokens into a specified syntax.

3. **Parsing:**

To break a sentence down into its component parts of speech with an explanation of the form, function, and syntactical relationship of each part

- The step two of Front end in Compilation
- Every programming Language has RULES that prescribe the syntactic structure (i.e. the way in which words and phrases can be organized to form sentences.)
- These rules can be formally specified using :

CONTEXT FREE GRAMMARS

(or GRAMMAR for Short)

A.k.a

BACKUS – NAUR FORM (BNF)

Intro to Context Free Grammar (CFG)



- Essential formalism for describing the structure of the programs in a programming Language
- A recipe for constructing elements of a set of strings of symbols (or tokens)
- Consists of a set production rules and a start symbol
- Each production rule:
 - Defines a named syntactic construct using Two parts
 1. An LHS (with a syntactic Construct) and
 2. An RHS (showing the possible form of the construct)
 - Connected by an Arrow symbol (meaning “ is defined as”)

sentence $\longrightarrow$ *subject predicate* ‘.’

Importance of the Grammar

Grammar offers significant advantages to both language Designers and Compiler writers:

1. Gives precise, yet easy-to-understand, syntactic specification of Programming Languages
2. Helps construct efficient parsers automatically (not in all cases – of course)
3. Provides a good structure to the language which in turn helps generate correct object code
4. Makes language open ended easily amenable to add additional constructs at a later date

Problem Statement:

- To find a derivation sequence in a grammar **G** for the input token stream
- or
- To conclude that none exists

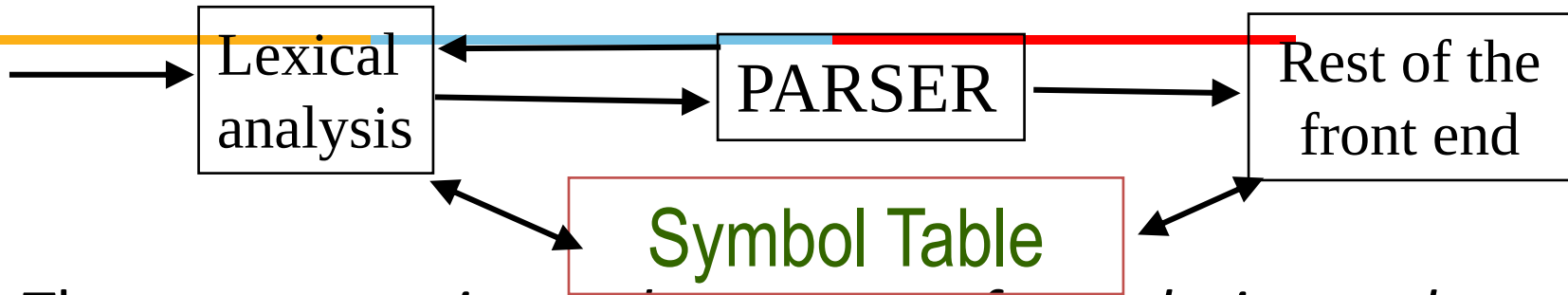
Solution

- Build parse tree for the string (of tokens) based on grammar rules
- The process is known as **P A R S I N G**

Constructs the syntax tree for a given sequence of tokens using grammar rules and productions such that:

1. The top node is labeled with the start symbol of the grammar
2. Leaf nodes are labeled with terminals and inner nodes with nonterminals
3. the children of the inner node labeled N correspond to the members of an alternative of N, in the same order as they occur in that alternative
4. The terminals labeling the leaf nodes correspond to the sequence of tokens, in the same order they occur in the input

The Role of a Parser



- The *parser continues the process of translation and validation* started by the Lexical Analyzer:
 1. Analyzing the phrase structure of the program,
 2. Adding hierarchical (tree) structure to the flat stream of tokens produced by the lexical Analyzer,
 3. Outputting a data structure conveying the program's syntax to subsequent compile phases, **AND**
 4. Reporting errors when the program does not match the syntactic structure of the source language

Parsing Methods

- Three Types of parsing methods in Vogue

1. Universal Parsing method

- Too inefficient to use in any practical compilers (hence not discussed any further)
- EX: Coocke–Younger–Kasami (CYK) algorithm

2. Top – Down Parsing

- Can be generated automatically or written manually
- Ex: : Left-to-Right –Top –Down Parser (A.k.a. **LL parser**)

3. Bottom – Up parsing

- Can only be generated automatically
- Ex : Left-to-Right –Bottom-Up Parser (A.k.a. **LR Parser**)

Top – Down & Bottom – Up Methods



- While they are efficient;
- They can work only on restricted classes of Grammars,
(such as LL and LR Grammars)

BUT

- They are expressive enough to describe most syntactic constructs in Programming Languages.

SO

- Care must be taken while defining the grammar for the language to make sure that the grammar is *unambiguous*

TOP – DOWN Parsing



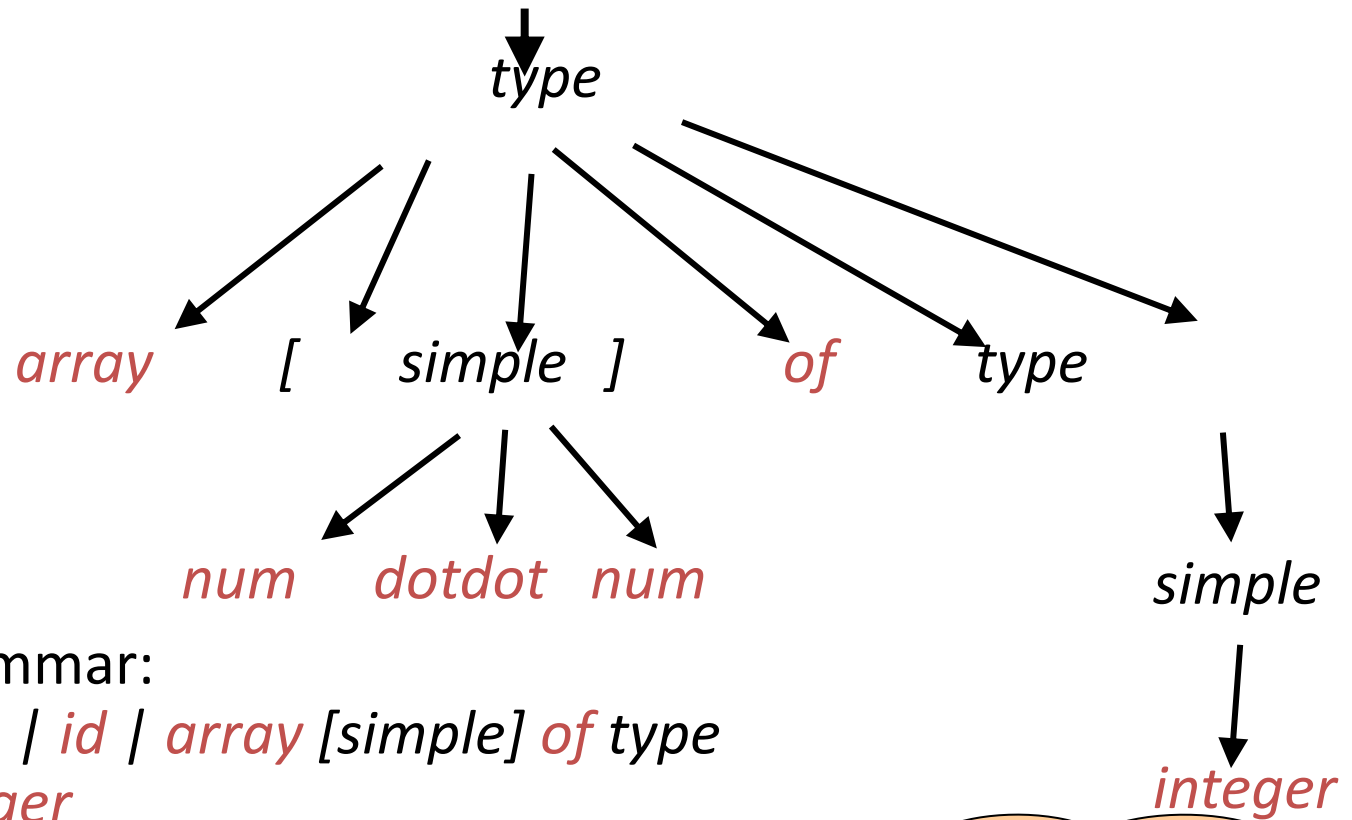
- An attempt to construct a parse tree for an input string from the root and creating the nodes in PRE-ORDER (i.e. the top of the tree is constructed before its nodes)
- Can also be viewed as an attempt to find a leftmost derivation for an input string
- The way it works:
 1. The process starts at root node say N
 2. Repeat until the fringe of the parse tree matches the input string
 1. Determine the correct alternative for N
 2. Parser then proceeds to construct the left child of N
 3. The process of determining the correct alternative for the leftmost child continues till the left most child is a terminal

The key Step

Top Down parsing example



String to be parsed :array [num dotdot num] of integer



Consider the Grammar:

$type \rightarrow simple \mid id \mid array [simple] of type$

$simple \rightarrow integer$

$\mid char$

$\mid num \ dotdot \ num$

Token dotdot “..” is used to stress that character sequence is treated as unit

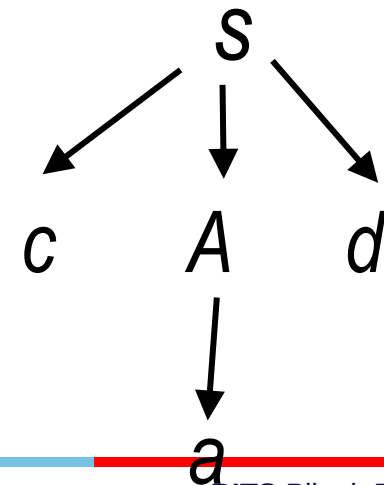
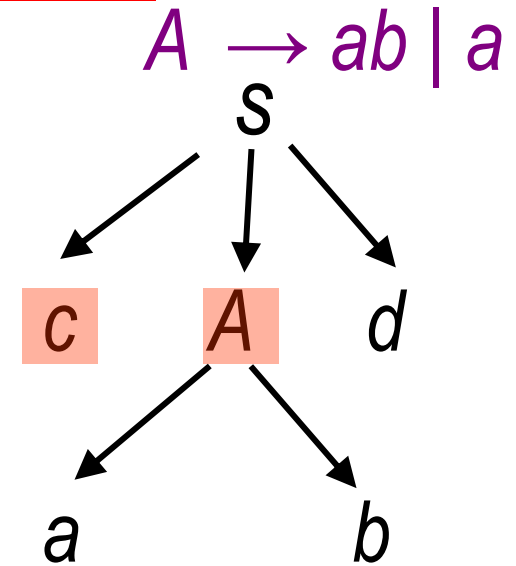
Top Down Parsing – Another Case



- Consider the grammar : $S \rightarrow cAd$
- String to be parsed : $W = c a d$

■ Parse tree Construction would be:

1. Start with a single node S
2. Use first production to expand S
3. Expand the nonterminal A using the first alternative from rule 2 as we have a match for input token a
4. But b does not match d ;
5. We have to **BACKTRACK** to step 3 and try second alternative from rule 2



Top Down Parsing - Observations

- In general,
 1. The selection of a production for a nonterminal may involve trial – and – error
 2. We may have to try a production and backtrack to try another production if the first is found to be unsuitable.(A production is unsuitable ,if after using the production, we can not complete the tree to match the input string)
- If we can make the correct choice by looking at just the next input symbol, we can build a *Predictive Parser* that can perform a top-down parse without backtracking

Top Down Parsing - Observations

- In general,
 1. The selection of a production for a nonterminal involve trial – and – error
 2. We may have to try a production and backtrack to try another production if the first one is found to be unsuitable.(A production is unsuitable if after using the production, we can not complete the parse to match the input string)
- If we can find the correct choice by looking at just the next input symbol, we can build a *Predictive Parser* that can top-down parse without backtracking

Programming language grammars are often suitable for Predictive Parsing

Predictive Parsing



- Most common technique used in parsers (Particularly TRUE in case of Parsers written manually)
- Use grammars that are carefully written to eliminate left recursion
- Further, if many alternatives exist for a nonterminal, the grammar is so defined that the input token will uniquely define the one and only one alternative and that will lead to correct parsing or Error.
- No question of backtracking and trying another alternative

Predictive Parsing Example



Consider the grammar:

```
stmt → if expr then stmt else stmt  
      | while expr do stmt  
      | begin stmt_list end
```

A parser for this grammar can be written with the following simple structure:

Based only on the first token, the parser knows which rule to use to derive a statement, *because all the three outcomes are unique even at the first letter.*

Therefore this is called a
Predictive Parser.

```
switch(gettoken())  
{  
    case if:  
        ...  
        break;  
    case while:  
        ...  
        break;  
    case begin:  
        ...  
        break;  
    default:  
        reject input;  
}
```

Parsing Table



- Represents a Grammar as
- A two dimensional array $M [A, \alpha]$, where:
 - A is the nonterminal and
 - α is the input symbol
- Used in Parser as the reference table to decide the possible values (thereby the choice of production to be used)

Grammar:

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \varepsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \varepsilon$
 $F \rightarrow (E) \mid id$



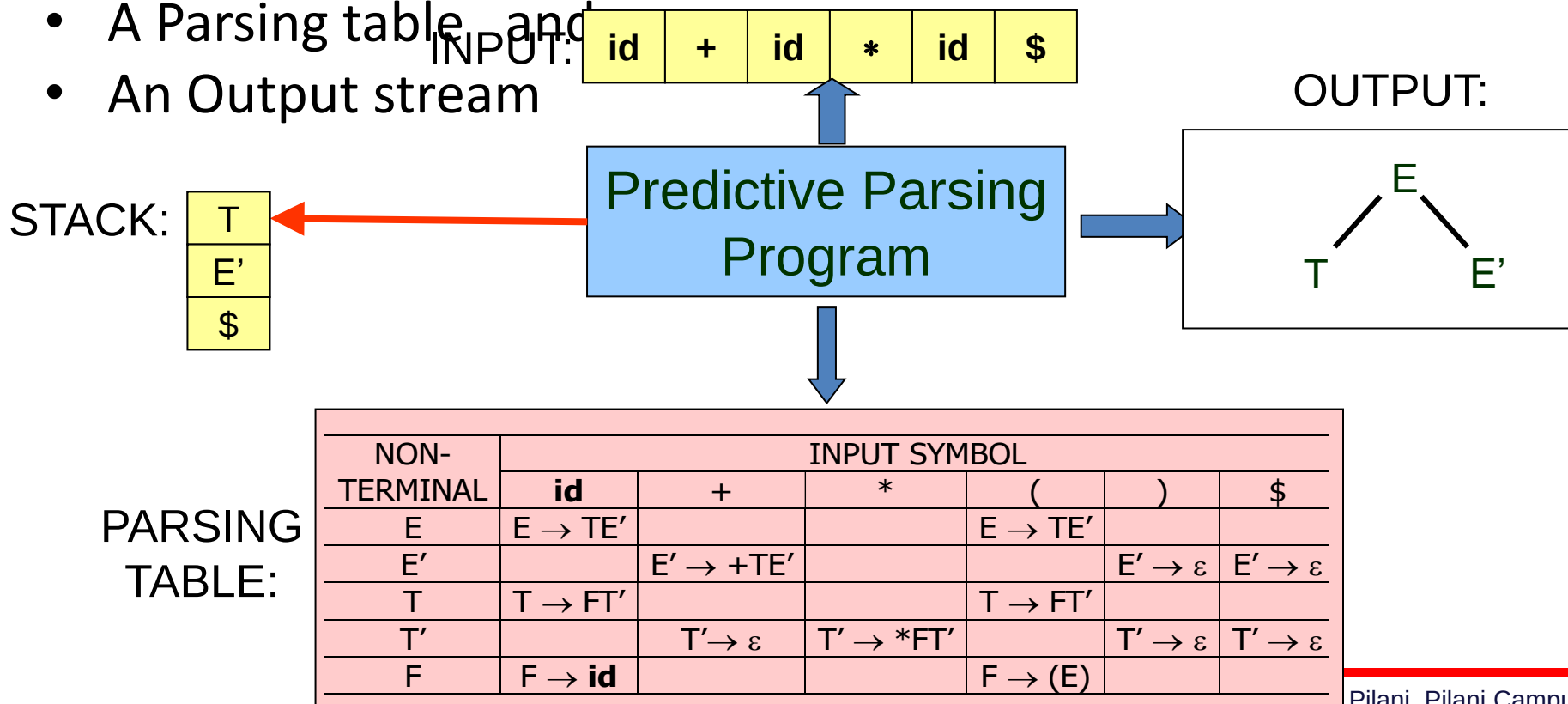
Parsing
Table:

NON- TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

A Table Driven Predictive Parser Program



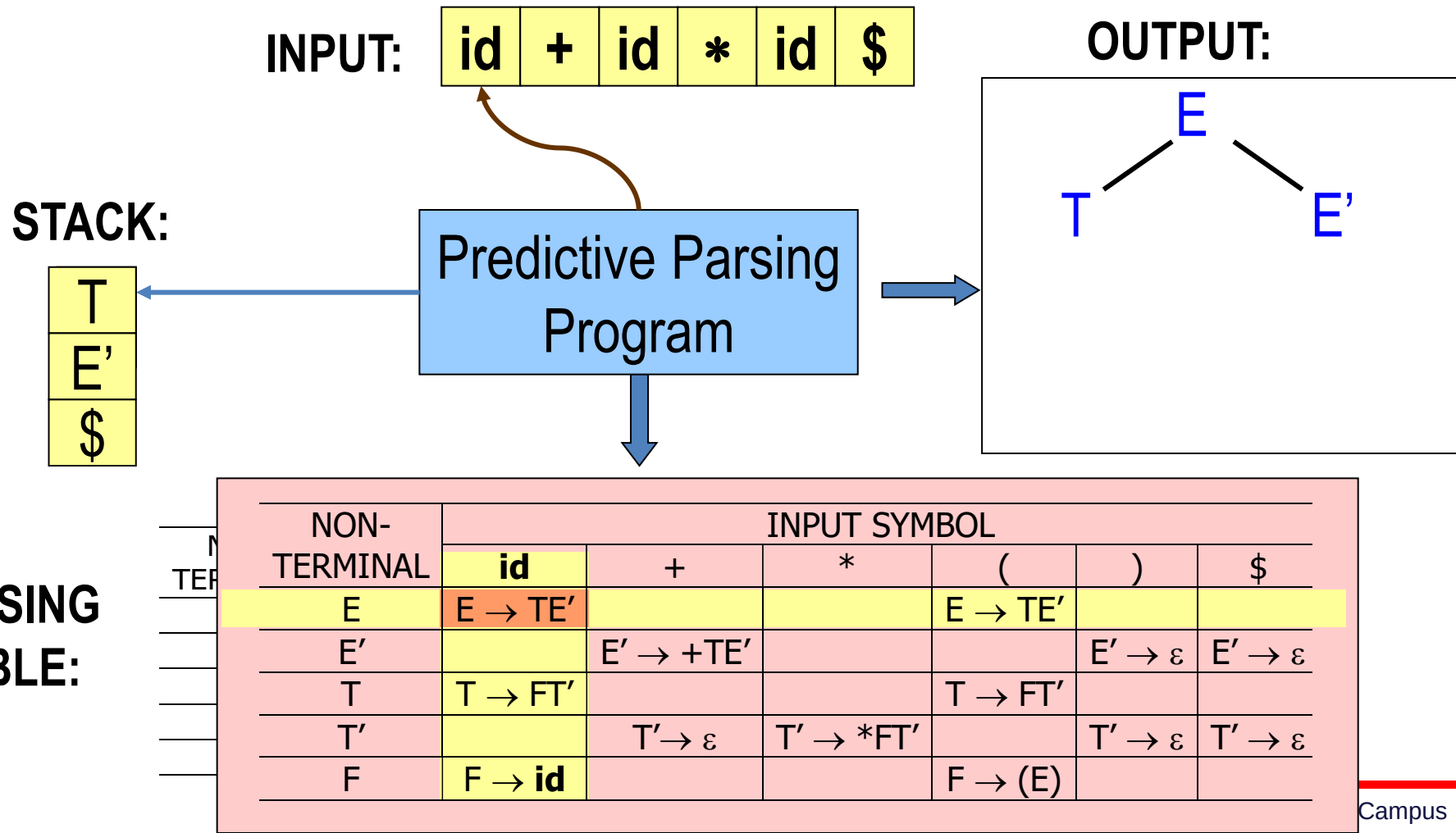
- An input Buffer (containing string to be parsed with \$ in the end)
- A stack (containing a sequence of grammar symbols with \$ at the bottom)
- A Parsing table and
- An Output stream



A Predictive Parser



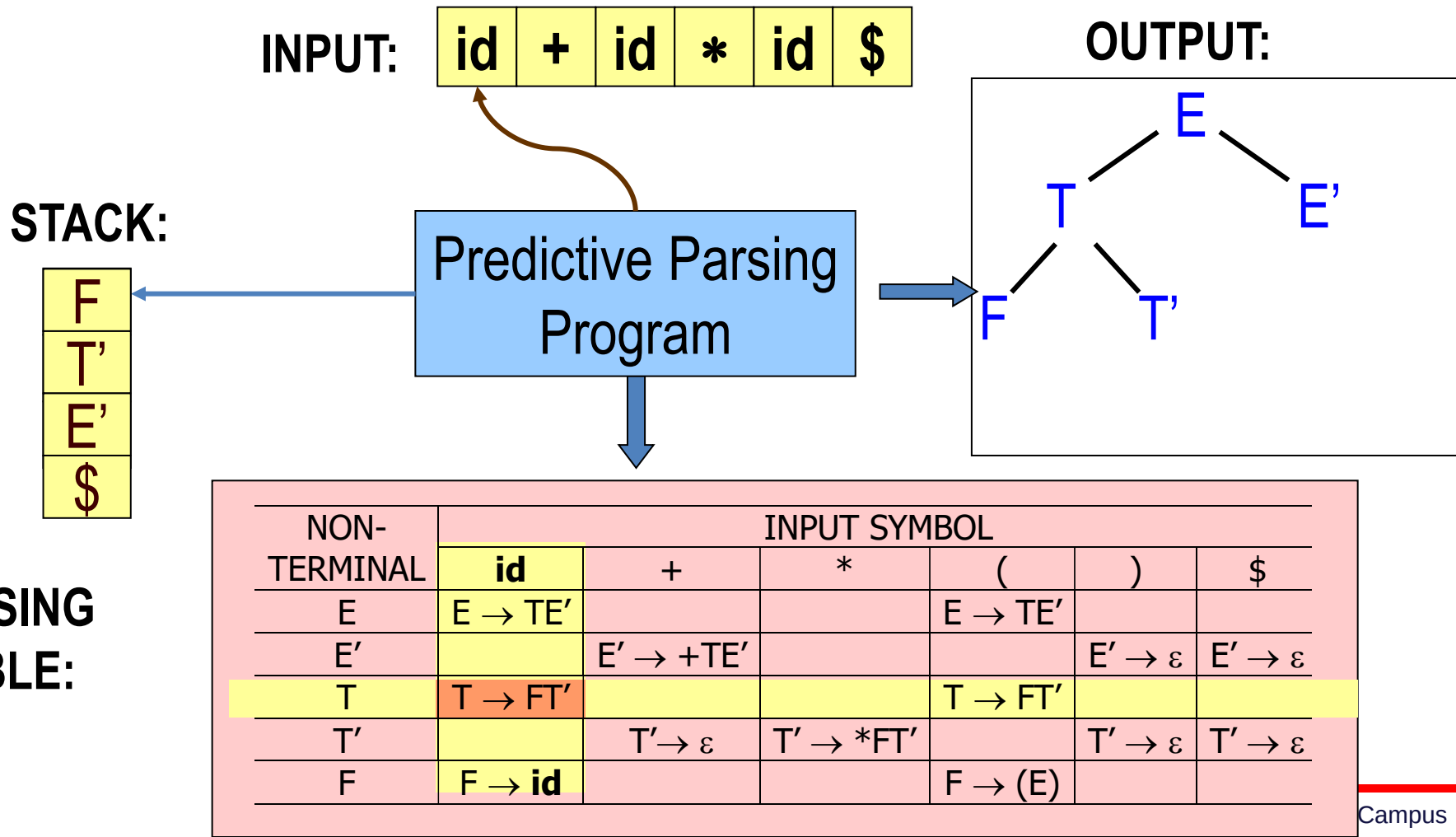
■ The Working of the Table driven Predictive Parser



A Predictive Parser



■ The Working of the Table driven Predictive Parser

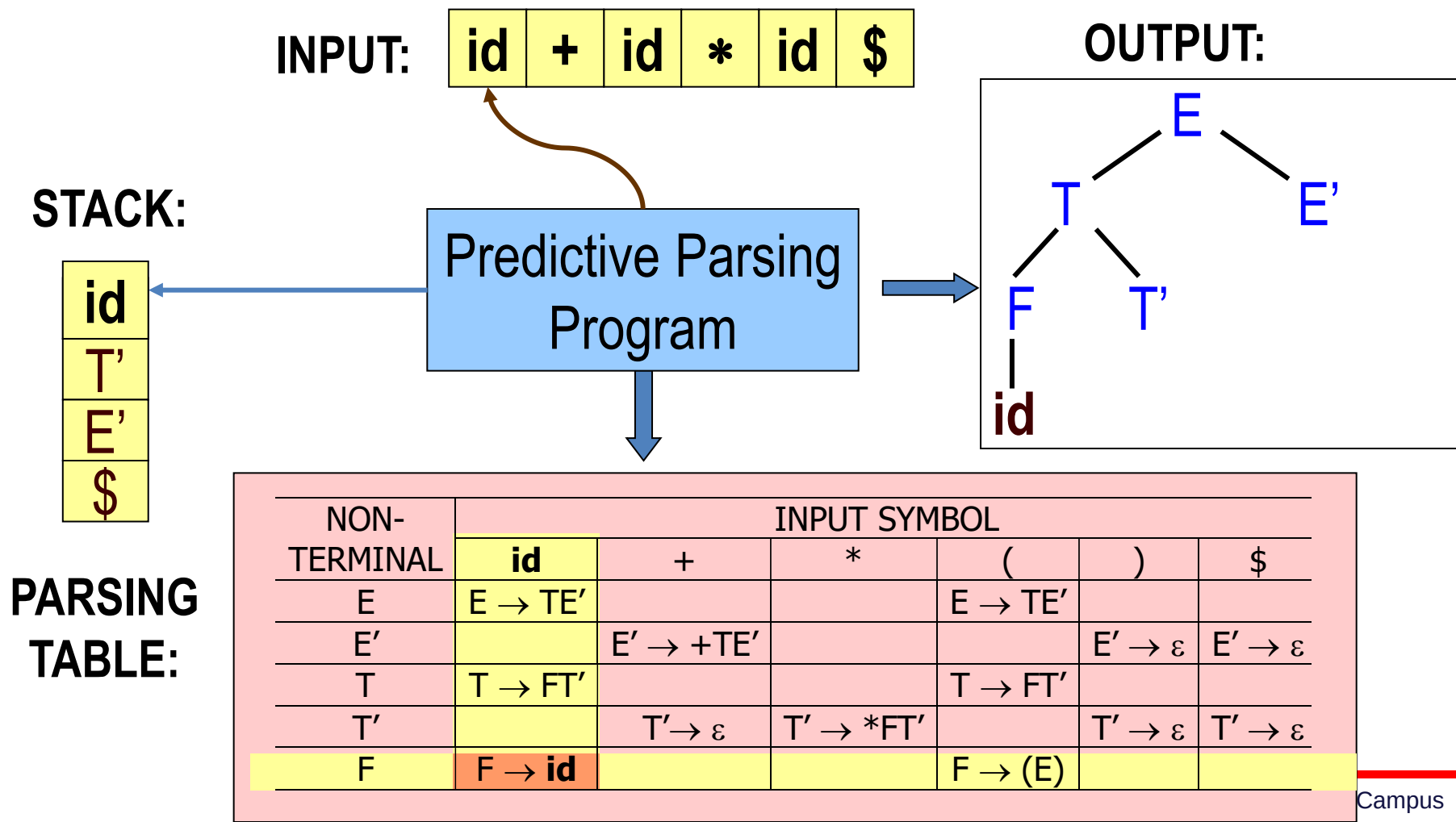


A Predictive Parser



■ The Working of the Table driven Predictive Parser

Action when $\text{Top}(\text{Stack}) = \text{input} \neq \$$: Pop stack, advance input.

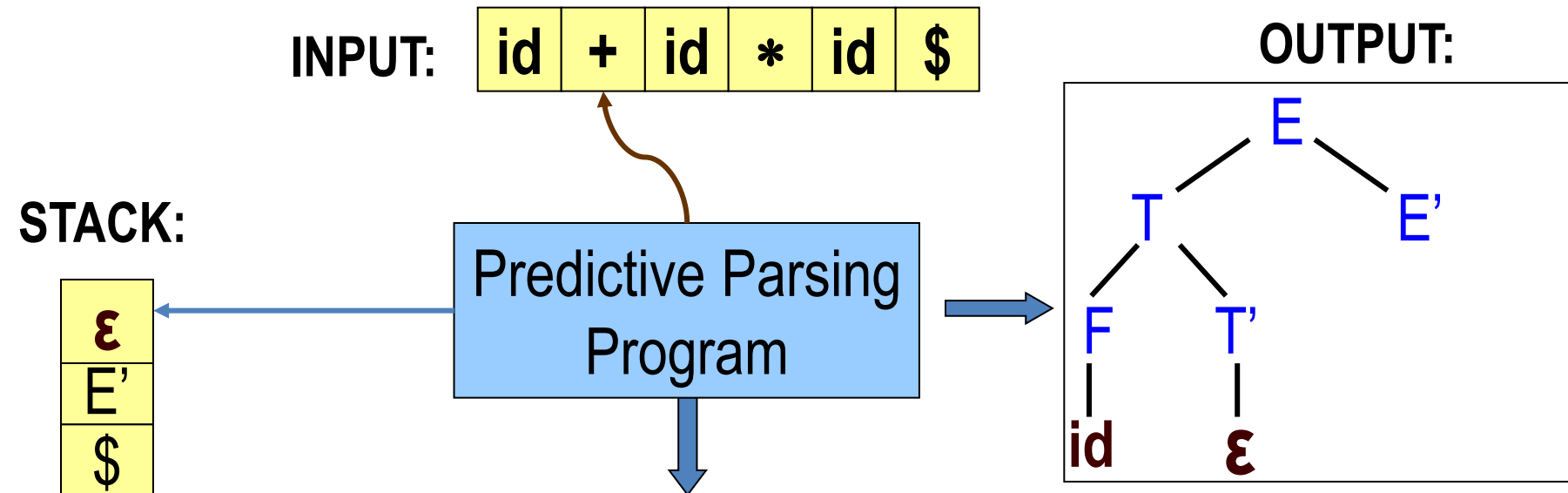


A Predictive Parser



■ The Working of the Table driven Predictive Parser

Action when $\text{Top}(\text{Stack}) = \epsilon$: Pop stack



**PARSING
TABLE:**

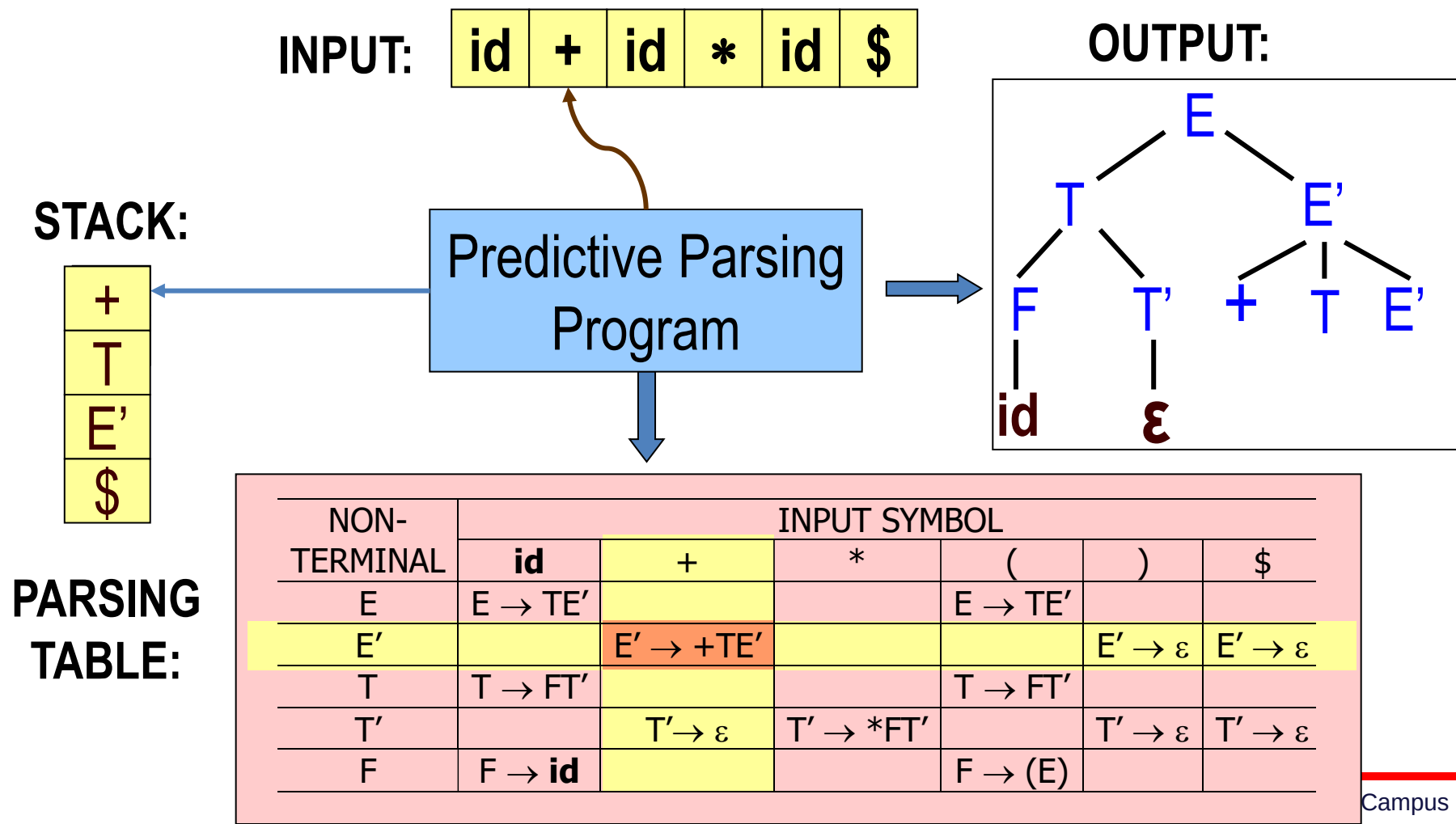
NON- TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

A Predictive Parser



■ The Working of the Table driven Predictive Parser

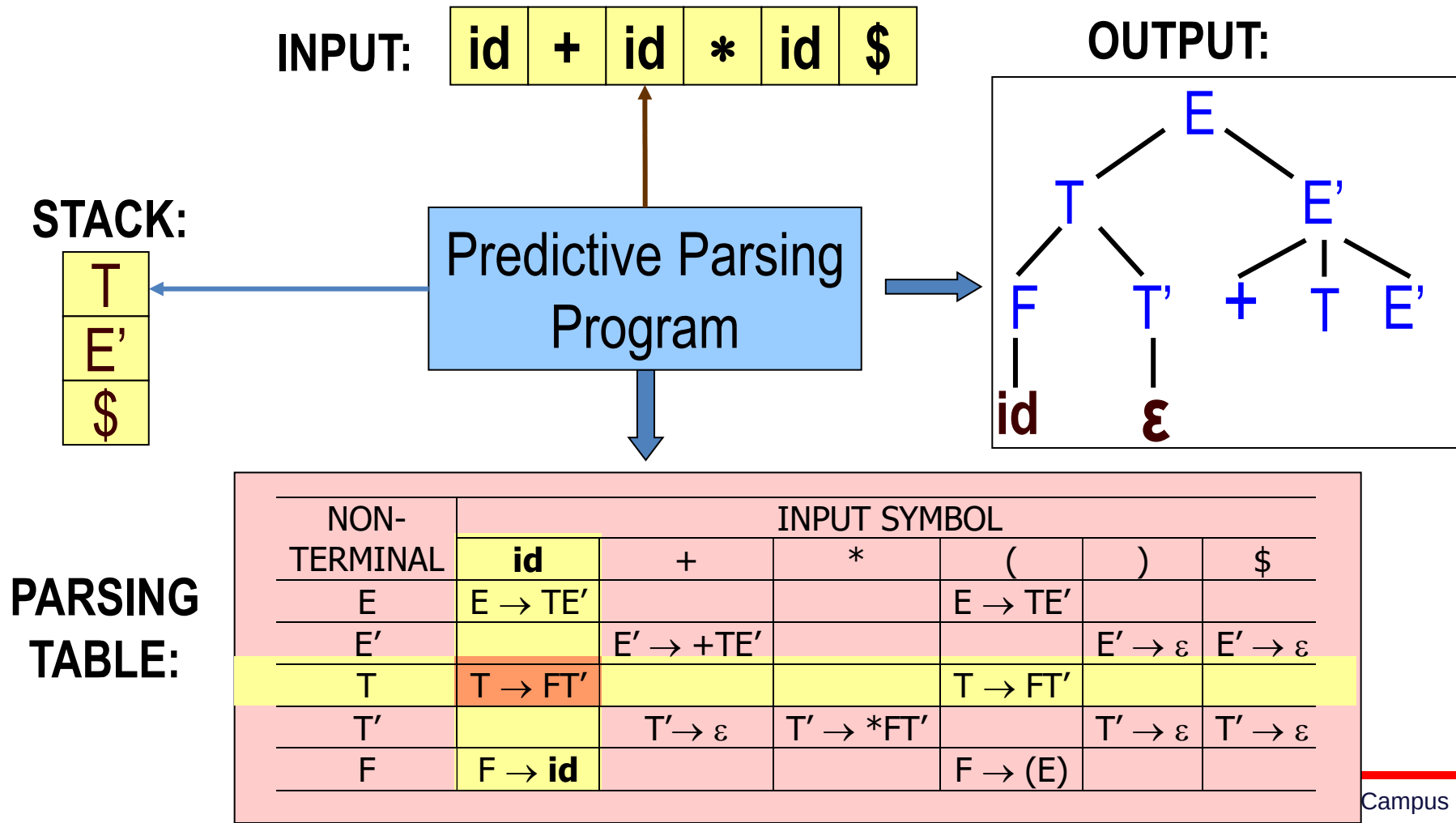
Action when $\text{Top}(\text{Stack}) = \text{input} \neq \$$: Pop stack, advance input.



A Predictive Parser



- The Working of the Table driven Predictive Parser



A Predictive Parser



The predictive parser proceeds in this fashion emitting the following productions:

$T \rightarrow FT'$

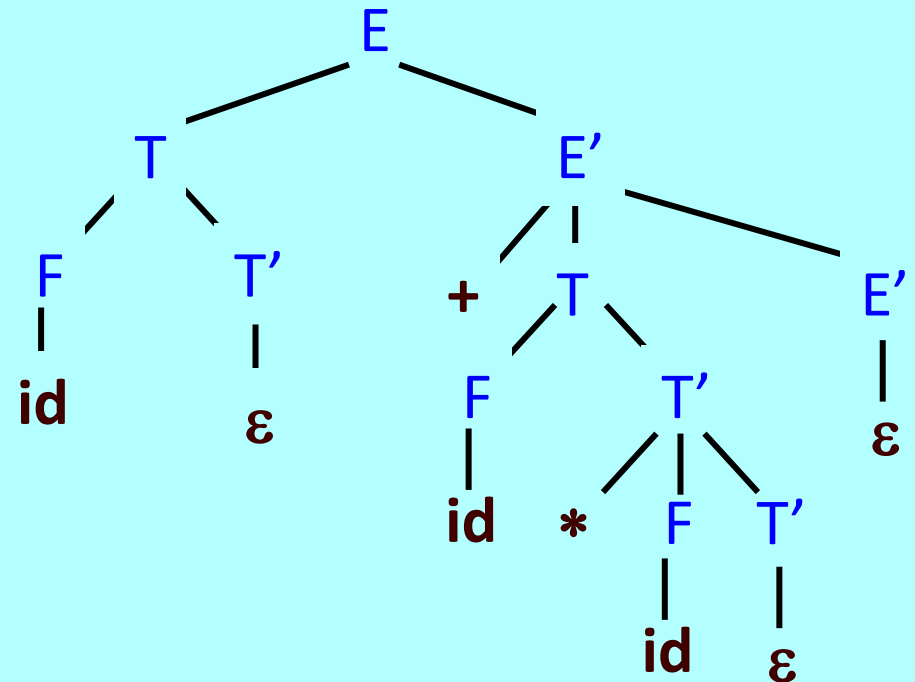
$F \rightarrow \text{id}$

$T' \rightarrow * FT'$

$F \rightarrow \text{id}$

$T' \rightarrow \epsilon$

$E' \rightarrow \epsilon$



When $\text{Top}(\text{Stack}) = \text{input} = \$$ the parser halts & accepts the input string – (id + id * id)

Algorithm for Predictive parser



- The program execution is controlled by TWO inputs
 - The input symbol a and the symbol on the top of the stack X
- There are THREE possibilities for the parser
 - If $X = a = \$$: halt & announce the successful completion of parsing
 - If $X = a \neq \$$: Pop X off the stack and advance the input pointer to next input symbol
 - If X is a nonterminal : Consult entry $M [X, a]$ in the Parsing table M and replace X on top of the stack with the RHS of the production
 - If $M [X, a]$ is error call error routine

Algorithm for Predictive parser

- The program execution is controlled by TWO :
 - The input symbol a and the symbol on top of the stack X
- There are THREE possibilities for the action :
 - If $X = a = \$$: halt & announce the completion of parsing
 - If $X = a \neq \$$: Pop X from the stack & advance the input pointer to next input symbol
 - If $X \neq a$: Consult entry $M[X, a]$ in the Parsing Table. If the entry is non-empty, push the symbol on top of the stack with the RHS of the entry.

This parser is also known as LL (1) Parser since

1. It parses **from left to right**, and does a **leftmost-derivation**
2. It looks up **1 symbol ahead** to choose its next action.

If the entry $M[X, a]$ is error call error routine

FIRST & FOLLOW



- The two functions represent a set of terminal symbols, that aid us in constructing Predictive parser by helping us fill the Parsing Table with valid entries
- Set of terminal symbols yielded by *FOLLOW* function can also be used in error recovery
- *FIRST* – if α is the string of grammar symbols then $FIRST(\alpha)$ is the set of terminals that begin the strings derived from α

(If $\alpha \xRightarrow{*} \epsilon$, then ϵ is also in $FIRST(\alpha)$)

- *FOLLOW(A)* – the set of terminals a that can appear immediately to the right of A such that there exists a derivation of the form

$S \xRightarrow{*} \alpha A a \beta$ for some α and β

FIRST & FOLLOW (Contd.)



- The set of terminal symbols (including ϵ) that can appear at the far left of any parse tree derived from a particular nonterminal is the FIRST set of that nonterminal
- The set of terminal symbols (including ϵ) That can follow a nonterminal in some derivation or the other, is called the FOLLOW set of that nonterminal
- A set of rules are followed to compute the FIRST and FOLLOW set
- These sets will be used in creating Parsing table

Rules to Create FIRST



FIRST rules:

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$
2. If $X \rightarrow \varepsilon$, then $\varepsilon \in \text{FIRST}(X)$
3. If $X \rightarrow aABC$, then $a \in \text{FIRST}(X)$
4. If $X \rightarrow ABCD$, then $\text{FIRST}(A) \in \text{FIRST}(X)$.
- 4a. Further, If $A \rightarrow \varepsilon$, in the above production, then $\text{FIRST}(B) \in \text{FIRST}(X)$ [& so on recursively]

GRAMMAR:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

SETS:

$$\text{FIRST}(\text{id}) = \{\text{id}\}$$
$$\text{FIRST}(*) = \{*\}$$
$$\text{FIRST}(+) = \{+\}$$
$$\text{FIRST}(() = \{($$
$$\text{FIRST}() = \{) \}$$
$$\text{FIRST}(E') = \{\cancel{\varepsilon}\{+, \varepsilon\}$$
$$\text{FIRST}(T') = \{\cancel{\varepsilon}\{*, \varepsilon\}$$
$$\text{FIRST}(F) = \{(, \text{id}\}$$
$$\text{FIRST}(T) = \text{FIRST}(F) = \{(, \text{id}\}$$
$$\text{FIRST}(E) = \text{FIRST}(T) = \{(, \text{id}\}$$



FIRST(E') = {+, ε}

FIRST(T') = {*, ε}

FIRST(F) = {(, id}

FIRST(T) = {(, id}

FIRST(E) = {(, id}

Rules to Create FOLLOW

FOLLOW rules:

GRAMMAR:

E → TE'
E' → +TE' | ε
T → FT'
T' → *FT' | ε
F → (E) | id

SETS:

FOLLOW(E) = {~~\$~~} {), \$ }

FOLLOW(E') = {), \$ }

FOLLOW(T) = { ~~), \$~~ {+,), \$ }

FOLLOW(T') = { +,), \$ }

**FOLLOW(F) = {~~+,), \$~~
{+, *,), \$ }**

1. If **S** is the start symbol, then **\$ ∈ FOLLOW(S)**

2. If there is a production **A → αBβ**, then,
EVERYTHING in **FIRST(β)** except **ε** is placed
in **FOLLOW(B)**

3. If there is a production **A → αB** then,
EVERYTHING in **FOLLOW(A)** is in **FOLLOW(B)**

3a. If there is a production **A → αBβ** where
FIRST(β) contains **ε** (i.e. **β $\xRightarrow{*}$ ε**) then,
EVERYTHING in **FOLLOW(A)** is in **FOLLOW(B)**

**A and B are non-terminals,
α and β are strings of grammar symbols**

Rules to Build Parse Table



FOLLOW SETS:

GRAMMAR:

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

FIRST SETS:

$FIRST(E') = \{+, \epsilon\}$
 $FIRST(T') = \{*, \epsilon\}$
 $FIRST(F) = \{(, id\}$
 $FIRST(T) = \{(, id\}$
 $FIRST(E) = \{(, id\}$

$FOLLOW(E) = \{), \$\}$
 $FOLLOW(E') = \{), \$\}$
 $FOLLOW(T) = \{+,), \$\}$
 $FOLLOW(T') = \{+,), \$\}$
 $FOLLOW(F) = \{+, *,), \$\}$

Rules

1. If $A \rightarrow \alpha$, if $a \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
2. If $A \rightarrow \alpha$, if $\epsilon \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal $b \in FOLLOW(A)$,
3. If $A \rightarrow \alpha$, if $\epsilon \in FIRST(\alpha)$, and $\$ \in FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$

PARSING TABLE:

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Predictive Parsing



- A top down method of parsing (*Recursive descent parsing*) in which a set of procedures are executed recursively to process the input.
- The speciality of predictive parsing is that :

The look ahead symbol unambiguously determines the procedure selected for each nonterminal
(No Prediction in fact !!!)

- Hence:

The sequence of procedures called implicitly defines a parse tree for the input.

```
procedure match (c : token);  
{  
if ( lookahead == c) then lookahead := nexttoken;  
  else error;  
}
```

```
procedure type;  
{if ( lookahead is [ integer, char, num ] ) then simple;  
  else if (lookahead == '↑') { match ('↑'); match (id); }  
  else if ( lookahead == array )  
    then { match (array); match ('['); simple;  
          match (']'); match (of); type; }  
  else error; }
```

procedure *simple*;

{

if (*lookahead == integer*) *match (integer)*;

else if (*lookahead == char*) *match (char)*;

else if (*lookahead == num*)

 { *match (num)*; *match (dotdot)*; *match(num*
);}

else *error*;

}

Error recovery in Predictive parsing



- Error is encountered by Predictive Parser when:
 - The terminal on the stack does not match the next input symbol
 - The nonterminal A is on the top of the stack, A is the input symbol and the parsing table is empty for entry M [A, a]
- Two methods used for recovery
 - 1. Panic – Mode error recovery**
 - Skip symbols on the input until a token in a selected set of synchronizing tokens appears
 - Effectiveness depends on set of synchronizing tokens chosen
 - 2. Phased – level recovery**
 - Fill empty slots of parsing table with pointers to error Routines

LL (1) Parsing

- Grammars whose predictive parsing tables contain no multiple entries are called LL(1).
- LL(1)
 - Left-to-right parse of input string
 - Leftmost derivation of grammar
 - 1 symbol look ahead (we know only what the next token is)
- LL (k) : we know what the next k tokens are
- Every LL(1) grammar is an LL(2) grammar and so on

More Examples on LL(1)

Example2:

$S \rightarrow AaAb / BbBa$ **First and Follow**

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Pdn	First	Follow
$S \rightarrow AaAb / BbBa$	a,b	\$
$A \rightarrow \epsilon$	ϵ	a,b
$B \rightarrow \epsilon$	ϵ	a,b

LL(1) Parsing Table :

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

*Whether the above grammar can be used for LL(1) Parsing?

More Examples on LL(1)

Example3:

$S \rightarrow A/a$

$A \rightarrow a$

Example5:

$S \rightarrow aABb$

$A \rightarrow c / \epsilon$

$B \rightarrow d / \epsilon$

Example4:

$S \rightarrow aB / \epsilon$

$B \rightarrow bC / \epsilon$

$C \rightarrow cS / \epsilon$

Example6:

$S \rightarrow aSa / \epsilon$

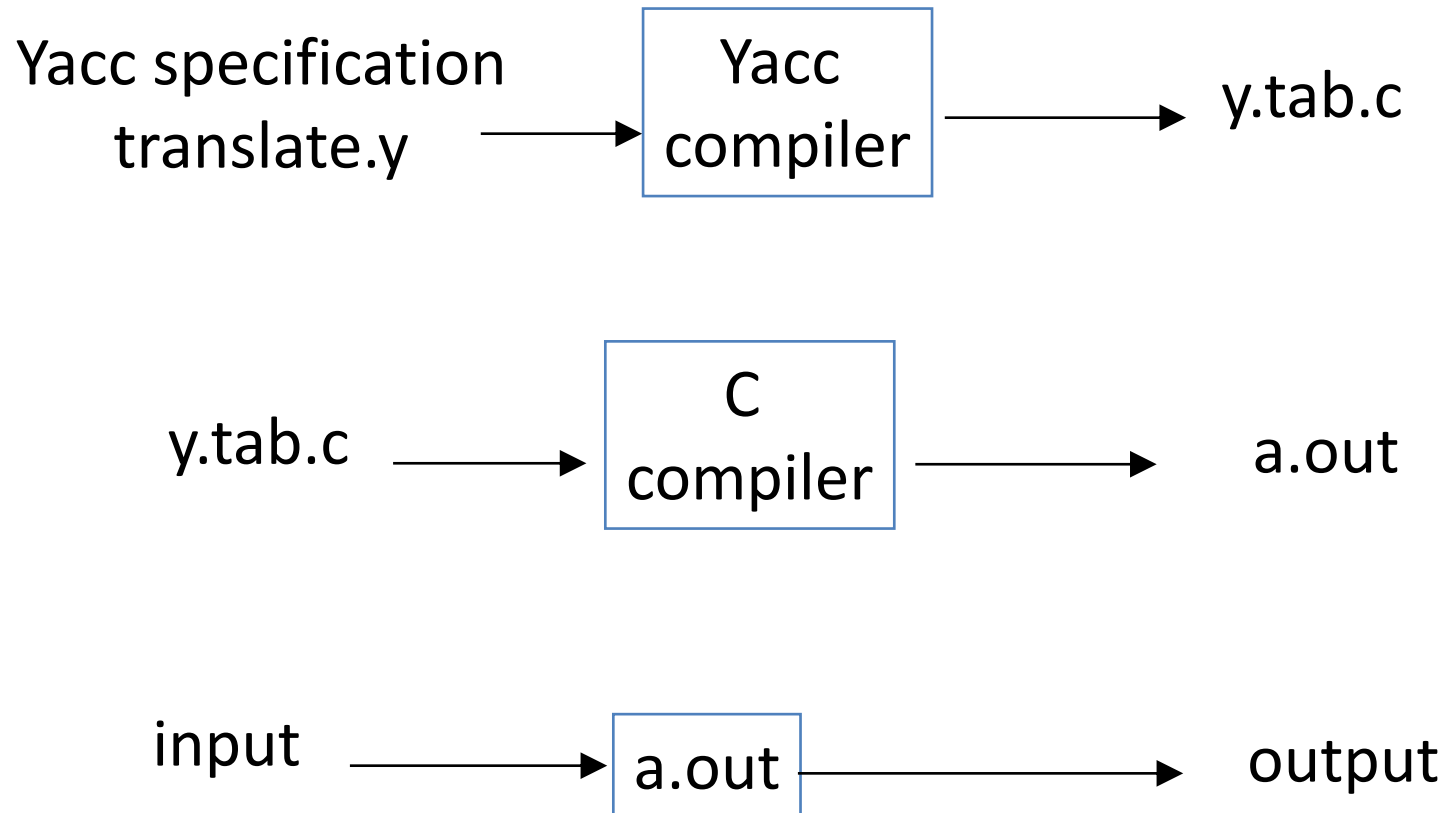
$A \rightarrow c / \epsilon$

Write the First and Follow and also the parsing table for the above examples to check the suitability of the grammar for LL(1)

Parser generators



Yet Another Compiler Compiler (YACC).



Sample program for Parser using Yacc



Lex Program: (ar.l)

```
%{
/* Definition section */
#include<stdio.h>
#include "y.tab.h"
extern int yylval;
}%
/* Rule Section */
%%
[0-9]+ {
    yylval=atoi(yytext);
    return NUMBER;

}
[\\t] ; [\\n] return 0;
return yytext[0];
%%
int yywrap()
{ return 1; }
```

Sample program for Parser using Yacc



Yacc Program: (prog.y)

```
%{  
/* Definition section */  
#include<stdio.h>  
int flag=0;  
%}  
%token NUMBER  
%left '+' '-'  
%left '*' '/' '%'  
%left '(' ')'
```

Sample program for Parser using Yacc



```
/* Rule Section */
```

```
%%
```

```
ArithmeticExpression: E{  
    printf("\nResult=%d\n", $$);  
    return 0;  
};
```

```
E:E+'E' {$$=$1+$3;}
```

```
|E '-'E {$$=$1-$3;}
```

```
|E '*'E {$$=$1*$3;}
```

```
|E '/'E {$$=$1/$3;}
```

```
|E '%'E {$$=$1%$3;}
```

```
| '('E')' {$$=$2;}
```

```
| NUMBER {$$=$1;}
```

```
;
```

```
%%
```

Sample program for Parser using Yacc



```
//driver code
```

```
void main()
{
    printf("\nEnter Any Arithmetic Expression which have operations
    Addition,Subtraction, Multiplication, Division, Modulus and
    Round brackets:\n");
    yyparse();
    if(flag==0)
        printf("\nEnter arithmetic expression is Valid\n\n");
    }
    void yyerror()
    {
        printf("\nEnter arithmetic expression is Invalid\n\n");
        flag=1;
    }
```

Sample program for Parser using Yacc



Compile and Run:

We have to compile both lex and yacc files as given below:

lex ar.l

yacc -d prog.y // this produces y.tab.h,y.tab.c which should be used in compilation as shown below

cc lex.yy.c y.tab.c -ll

./a.out

Enter Any Arithmetic Expression which have operations Addition, Subtraction, Multiplication, Division, Modulus and Round brackets:

1+5

Result=6

Entered arithmetic expression is Valid

In Summary

- **LL Parsing** (A.k.a Recursive Descent parsing)
- LL stands for **L**eft-to-right scan of input & **L**eftmost derivation
- Start by expanding the start symbol and then expand out nonterminals based on the input, building the parse tree from the top down
- Works only on grammars with some restrictions. (LL Grammars with no left recursion and the first token uniquely identifies the Production)
- Can be easily built manually

Click to edit Master title style





Click to edit Master title style

Thank you